

Ethereum

- **Bitcoin is a decentralized application for money.**
- **Ethereum is a decentralized platform for *any* application.**

With **Bitcoin**, you can only write down debit and credit transactions: "Alice pays Bob 1 BTC."

- With **Ethereum**, you can write down debit/credit transactions, but you can *also* write down a set of rules in the notebook itself. For example: "If Alice pays Bob 1 ETH, and today is a Tuesday, then automatically send 0.1 ETH from Bob to Carol."

Ethereum Virtual Machine (EVM)

How can thousands of different computers, all over the world, run the same piece of code and be **100% certain** they all get the exact same result?

Think about your own computer. The result of a program can depend on the time of day, a random number, a file on your hard drive, or information you fetch from the internet. If every computer on the Ethereum network tried to run a program that said "fetch the price of gold from google.com," they might all get slightly different answers at slightly different times. The network would instantly fall out of consensus. The ledger would break.

The system needs to be perfectly **deterministic**. For a given input, the output must *always* be the same, no matter who runs it or when they run it.

Ethereum solves this problem with a brilliant concept: the **Ethereum Virtual Machine (EVM)**.

- **It's a Specification, Not a Physical Thing:** The EVM is not a piece of hardware. It's a detailed blueprint for a CPU. It's a set of rules. Anyone can use this blueprint to build their own software version of the EVM (and they do, in different programming languages like Go, Rust, and Java). As long as everyone's software version follows the rules exactly, they are guaranteed to produce the same output.
- **It's Completely Isolated (Sandboxed):** Code running inside the EVM has no access to the outside world. It cannot access the computer's network, its file system, or other processes. It lives in a "walled garden." The only information it can "see" is information that is already on the blockchain itself (like the current block number, account balances, or data stored by other contracts). This is the key to achieving determinism.
- **It has a Simple Instruction Set:** Just like a real CPU has basic instructions like ADD, COPY, STORE, the EVM has its own set of simple instructions called **opcodes**. When a developer writes a program in a high-level language like Solidity (Ethereum's most popular language), it gets compiled down into these simple, universal EVM opcodes. This is what is actually stored on the blockchain.

No Randomness, No External API Calls and No Floating-Point Math

GAS Fees:

- **Programmer A** writes very clean, efficient code to calculate the average of 10 numbers.
- **Programmer B** writes messy, inefficient code to do the same task. It has unnecessary steps and loops.

Imagine the user of that program is a bit optimistic and thinks the code is more efficient than it is. They set the **Gas Limit** on their transaction to **50,000**. What happens to their transaction, and what happens to the fee they paid?

Two Types of Account in Ethereum

1. Externally Owned Accounts (EOAs)

- **What it is:** This is the account you and I would create with a wallet like MetaMask or Ledger.
- **How it's controlled:** It is controlled by a private key. Only the person with the private key can sign and authorize transactions from this account.

- **What it can do:** It can hold and send Ether (ETH). It can also initiate transactions to interact with other accounts (both EOAs and contracts).
- **Key Property:** An EOA **cannot** have code associated with it. It is passive. It only reacts when its owner tells it to do something.

2. Contract Accounts

- **What it is:** This is an account that is created when a developer deploys a smart contract to the network.
- **How it's controlled:** It is controlled by its own internal **code**. It has no private key. It "wakes up" and executes its code when it receives a transaction from an EOA or a message from another contract.
- **What it can do:** It can hold and send ETH. It can do anything its code tells it to do: run complex calculations, read/write its own storage, and even call other contract accounts.
- **Key Property:** A Contract Account is an autonomous agent living on the blockchain. It's a robot waiting for a command. This is where the code actually lives.

Key part of account

- **Nonce:** A counter that goes up by one every time the account *sends* a transaction. This is a security feature to prevent someone from taking one of your old transactions and "replaying" it to send money again.
- **Balance:** The amount of Ether (the native currency) this account owns, stored as a simple number.
- **codeHash:** If this row represents a smart contract, this column contains the hash of its code. If it's a regular user account (an EOA), this column is empty. This is how the network distinguishes between users and code.
- **storageRoot:** If this is a smart contract, it has its own little private database or "storage." This column contains the root hash of that storage, which is a fingerprint of all the data the contract is holding.

How to send Transaction

- from: Your Address
- to: Your Friend's Address
- value: 1 ETH
- gasLimit: 21,000 (standard for a simple transfer)
- gasPrice: 20 Gwei
- nonce: 74 (your account's next transaction number, fetched from the network)

Where the transaction is stored and where the balance is stored?

The **Ethereum state** refers to the current status of **all accounts and smart contracts** on the Ethereum blockchain at a specific point in time.

Old State + Transactions = New State

The "accounts" are not stored inside the blocks. They are stored in a separate, massive database on each node's hard drive. The block only contains a cryptographic *proof* (the stateRoot) that this database was updated correctly.

Database 1: The Blockchain (The Journal)

- **What it is:** A simple, linear list of blocks.
- **How it's stored:** This is literally a set of files on your hard drive, one after another (block_0, block_1, block_2, ...). Each block file contains its header and the list of all transactions included in it.
- **How it grows:** It only ever grows. You just keep appending new blocks to the end of the chain. It's an append-only log. This is relatively simple data storage.

Database 2: The State (The Current Balance Sheet)

- **What it is:** The massive "spreadsheet" of all accounts and their properties (nonce, balance, storage, code). This is a much more complex data structure.
- **How it's stored:** This is stored in a highly optimized key-value database on your node's hard drive (commonly using a database like LevelDB). The "key" is the account address, and the "value" is the account's data. This database is structured as a **Merkle Patricia Trie**.

It's a special kind of database that has one magical property: you can generate a single, 32-byte hash (the stateRoot) that acts as a unique fingerprint for the *entire contents of the database*. If even one bit changes in any account's balance anywhere, the final stateRoot hash will change completely.

- **How it changes:** This database is constantly being **modified**. When a new block arrives, your node reads the transactions, and for each one, it updates this state database. It might decrease one account's balance, increase another's, and change a contract's storage. It's a living, breathing database.

A node never blindly trusts its own local database. It constantly verifies its local database against the cryptographic commitments recorded on the blockchain.

- previousBlockHash: 00000000
- transactions: [Tx A]
- StateRoot
- Digital Signature of Validator
- Address of Validator
- Reward
- number

Rule of 2/3

Can a smart contract *start a transaction by itself*?

For example, could a Decentralized Finance (DeFi) lending protocol automatically send you your daily interest payment at exactly 8:00 AM every day, without anyone prompting it?

Smart Contract and a Token

Solidity Contract:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0; contract SimpleCounter {

    // State variable to store the counter value
    uint256 public count;

    // Constructor - runs once when contract is deployed
    constructor() { count = 0;

    // Initialize counter to 0
    }

    // Function to increment the counter by 1
    function increment() public { count += 1; }

    // Function to decrement the counter by 1
    function decrement() public { require(count > 0, "Counter cannot go below zero"); count -= 1; }

    // Function to get current count (already public, so this is optional)
    function getCount() public view returns (uint256) { return count; }

    // Function to reset counter to zero
    function reset() public { count = 0; }

    // Function to set counter to specific value
    function setCount(uint256 _newCount) public { count = _newCount; } }
```

The Solution: The ERC-20 Token Standard

Instead of every developer inventing their own way to code a currency, the Ethereum community proposed a standard interface, a blueprint, called **ERC-20**

the contract has one primary variable in its storage: a

mapping

. You can think of this as a simple spreadsheet or a dictionary inside the contract itself.

mapping(address => uint256) public balances;

This line of code says: "Create a list where the key is an Ethereum address and the value is a number (the balance)."

- `totalSupply()`: A simple function that returns the total amount of the token that exists.
- `balanceOf(address who)`: A function that looks up the balances mapping and returns the balance for the address who. (This answers Question 1).
- `transfer(address to, uint256 amount)`: This is the key function. When you call it, the code does the following:
 - Checks if your address (`msg.sender`) has enough tokens in the balances mapping.
 - If yes, it subtracts the amount from your balance.
 - It adds the amount to the to address's balance.
 - If the check fails, it reverts the transaction.

ERC-20 is not a piece of software or a protocol. It is a technical *standard* or *specification*.

Think of it as a **blueprint** or a **universally agreed-upon set of rules** for creating a token on Ethereum.

- **Alice creates a token called "AliceCoin."** To check a balance, her function is named `getAliceBalance(address)`. To send tokens, her function is `sendAliceCoin(address, amount)`.
- **Bob creates "BobCoin."** His functions are `fetch_balance(address)` and `move_tokens(address, amount)`.
- **Carol creates "CarolCoin."** Her functions are `balanceOf(address)` and `do_transfer(address, amount)`.

The Solution: A Common Interface

The developers in the Ethereum community recognized this problem early on. A group of them wrote **Ethereum Request for Comments #20**. This document proposed a standard.

It said: "If you want to create a fungible token (where each token is identical, like a dollar bill), your smart contract **MUST** implement the following set of functions and events with these exact names and parameters."

ERC-20 is this mandatory public interface. It's a contract that a smart contract makes with the outside world.